

PROJECT AND TEAM INFORMATION

PROJECT TITLE

BMP Image Processing and Pixel Analysis System.

STUDENT / TEAM INFORMATION

<i>Team Name:</i>	
Team member 1 (Team Lead)	<i>Sharma, Ananya – 2510010367 ananyashar.10227@gmail.com</i> 
Team member 2	<i>Kaur, Gurveen – 2510011077 kaurgurveen211@gmail.com</i> 
Team member 3	<i>Jain, Rashika – 2510010842 rashika2122jain@gmail.com</i> 
Team member 4	<i>Shaily, Ashvath – 2510010076 ashvathshaily247@gmail.com</i> 

PROPOSAL DESCRIPTION

MOTIVATION

We use and edit images almost everyday but we usually do not think what happens inside a computer when an image is processed. A digital image is made up of pixels and each pixel contains color information that can be changed to apply different effects. We want to build a simple and lightweight BMP image processing system using C++ through this project. The user will be able to upload a BMP image, choose a desired operation such as grayscale, brightness, inversion, contrast, sepia, thresholding or RGB-channel adjustment and obtain a new processed image.

Our main motivation to build this is to understand how image processing actually works at a basic level instead of using readymade image processing functions. We want to learn how a BMP file is read, how its pixel information is stored, and how changing the RGB values of pixels changes the final result.

We also want to connect the C++ and Data Structures concepts we have learned with a practical application. We will use different data structures for different parts of the program such as maintaining the filter sequence, managing processing requests, and keeping track of previous image states for undo. Our project is not meant to replace professional editing software such as Adobe Photoshop. It is a smaller system designed to perform basic image processing while helping us understand the concepts behind it.

STATE OF THE ART / CURRENT SOLUTION

Nowadays a user can edit images using applications such as Photoshop, GIMP, Paint and other online image editing tools. These applications provide features such as changing brightness, applying filters, adjusting colors and converting images to grayscale.

These tools are mainly designed for users and usually hide the actual process happening. A beginner using them doesn't see how the image file is read or how individual pixel values are changed.

This project approaches the similar basic problem from a learning and implementation perspective. Instead of completely depending on an existing image processing library, we will work with the BMP file and its pixel data using C++ directly, in raw form. This will make us understand the connection between the image file, individual pixels, image processing operations and the final output. The system will focus only on selected basic operations rather than trying to provide all the features of a professional image editor.

PROJECT GOALS AND MILESTONES

Our main goal of this project is to create a working C++ program that can take a BMP image, process its pixels according to the user's choices and produce a new BMP image. This program will provide operations such as:

- *Grayscale*
- *Brightness adjustment*
- *Contrast adjustment*
- *Inversion*
- *Sepia*
- *Thresholding*
- *RGB-channel manipulation*
- *Connected-region detection using Graphs, BFS and DFS*

We'll use Data Structures as actual parts of the system rather than adding them just for demonstration.

- ***Linked List:*** *To maintain the order of filters selected by the user.*
- ***Queue:*** *To manage image processing requests.*
- ***Stack:*** *To maintain image history and support Undo.*
- ***Graph:*** *To represent the connectivity between neighbouring pixels/regions after thresholding.*

C++ Object-Oriented Programming will be used to divide the project into meaningful classes and make an easier to manage program.

Planned Milestones:

1. Phase 1:

Understand BMP format and pixel structure.

Implement BMP reading and writing.

Store and access pixel information.

2. Phase 2:

Implement basic image filters.

Implement Linked List filter pipeline.

Implement Queue for processing requests.

Implement Stack for image history and Undo.

Implement thresholding and identify neighbouring pixel connectivity.

3. Phase 3:

Implement Graph representation and BFS/DFS traversal for connected-region detection.

Combine the modules using C++ OOP.

Test the above program with different BMP images and filter combinations.

Complete the CLI, documentation, testing and final demonstration.

PROJECT APPROACH

We plan to make this project a command line application using C++. The user will provide the BMP image file as input and the program will read the image file to access its pixel information using binary file handling.

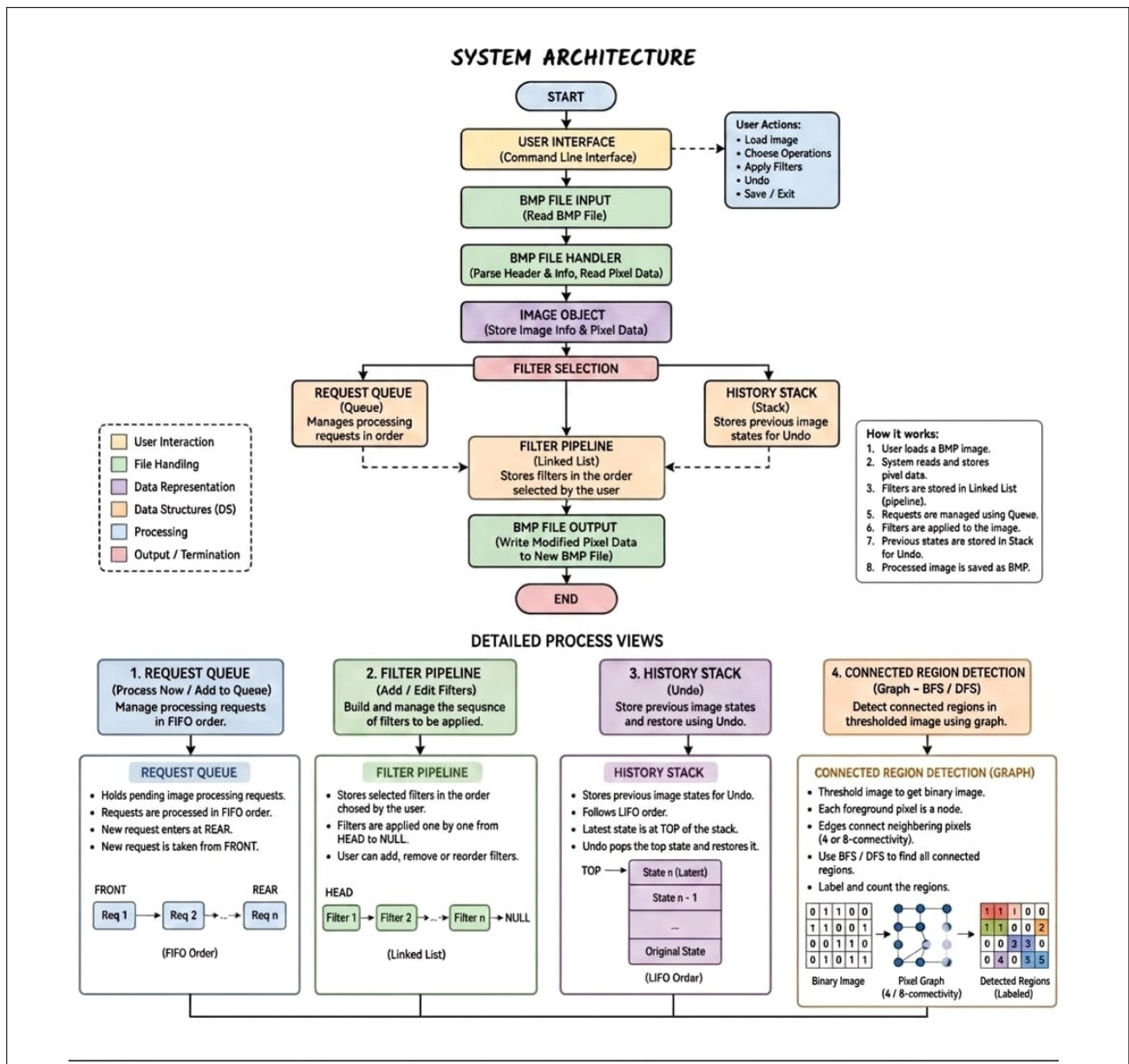
After loading the image the program will display a menu containing different options like grayscale, sepia, inversion, thresholding, and RGB-channel manipulation. The user can select one or more options depending on the operations they want on the image. The selected operations will be performed directly on the pixel values of the image. For example, brightness and contrast operations will change the corresponding pixel values while grayscale will convert the pixel information into the shades of gray.

Data Structures will be an important part of the program for image processing. A Linked List will be used to maintain and store the selected filters in order in which they need to be applied. A Queue will be used to manage image processing requests in the order they are received. While a Stack will store previous image states so that the recent changes can be undone by the user. A Graph will be used to represent the connectivity between neighbouring pixels after thresholding. BFS and DFS will be used to traverse those connected pixel regions and identify the separate connected regions in the image.

Use of Object Oriented Programming is to divide the project into separate classes based on their roles. For example, a pixel class can represent individual pixel data, separate Filter classes can manage different image processing operations. Data Structures related classes will be used to process filter pipeline, process requests and maintain image history. This will make the code organized and efficient to debug. After all operations are completed the modified pixel data will be written into a new BMP file which will be displayed as output.

Overall the project will consist of C++, OOP, Data Structures, file handling and basic image processing techniques into one application.

SYSTEM ARCHITECTURE (HIGH LEVEL DIAGRAM)



PROJECT OUTCOME / DELIVERABLES

By the end of this project we aim to have a working C++ command line application for basic BMP image processing. The final program will be able to:

- Take a BMP image as input.*
- Read and work with its pixel data.*
- Apply different image processing operations.*
- Apply multiple filters in a selected order.*
- Detect connected regions in a thresholded image using Graph traversal with the use of BFS and DFS.*
- Manage processing requests using a Queue.*
- Maintain previous image states using a Stack.*
- Provide an undo option.*
- Generate a new BMP image containing the changes.*

We will provide the complete source code, project documentation, system architecture, testing results and demonstration of a working program. The final project will bring together the concepts of C++ that we have learned along with concepts of Data Structures, file handling and basic image processing algorithms in one practical application.

ASSUMPTIONS

- 1. We will initially work with an uncompressed 24-bit BMP image.*
- 2. The user is expected to provide a valid BMP image.*
- 3. Standard C++ file handling will be used for reading and writing the BMP file.*
- 4. Image history for Undo will require additional memory depending on the image size.*

REFERENCES

References used are:

- 1. LearnCpp - Used for understanding deeper concepts of C++, classes, objects, file handling.*
- 2. GeeksForGeeks - Used for Data Structures for referring to concepts of Linked Lists, Stacks and Queues, Graphs, BFS and DFS.*
- 3. Book - Digital image processing by Rafael C. Gonzalez - Used to understand the basic digital image processing concepts and image processing techniques.*
- 4. Wikipedia - Used as reference for understanding the BMP file format and its basic structure.*